Stress + Chaos Test Design (Lava equivalent of HelixConstitution §11.4.85)

Status: Phase 1 scaffold landed (lava-api-go stress, host-runnable now). Later phases deferred. **Author:** design+investigation subagent, 2026-05-31 (68th-cycle follow-up). **Pin note:** This doc is written against constitution pin `208e2c8` AND the fetched upstream `origin/main` = `883ccc1` (which is where §11.4.85 lives). The pin is NOT bumped by this work (CONST-049 / operator-gated). §11.4.85 becomes mechanically binding when the pin advances; this scaffold lands the Lava equivalent ahead of that so the bump is not blocked on a missing suite.

---

1. The mandate — §11.4.85 verbatim

From `constitution/Constitution.md` at upstream `origin/main` (`883ccc1`), lines 7382-7396+ (identical text already present in the pinned tree at the same line offsets):

> ### §11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)
>
> **Short tag:** `stress-chaos-mandate`.
>
> **Forensic anchor (verbatim user mandate, 2026-05-24):**
>
> > "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"
>
> **The mandate.** Every fix or improvement landed in a consuming project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. A fix that PASSes its happy-path test but has never been exercised under stress or under fault-injection is a §11.4 / §107 PASS-bluff at the resilience layer: it claims to work but carries no evidence that real-world adversarial conditions (sustained throughput, parallel contention, partial failure, resource exhaustion, malformed input) leave the fix intact and the user-visible behaviour correct.

> **Definitions (closed-set, mechanically auditable):**
>
> 1. **Stress test** — exercises the fix-under-test under sustained or concurrent load above ordinary usage. At MINIMUM one of:
>   - **Sustained load** — N ≥ 100 sequential iterations OR ≥ 30 seconds wall-clock continuous load. Per-iteration latency MUST be recorded; percentile distribution (p50/p95/p99) MUST be reported.
>   - **Concurrent contention** — N ≥ 10 parallel invocations. All N MUST complete. No deadlock, no resource leak (file-descriptor count, process-table count, RSS), no data race in shared state.

(The clause continues with the chaos-test definition — fault injection, dependency kill, latency injection, malformed input, resource exhaustion — and the requirement that recovery after the chaos action be asserted on user-visible state. The full body lives in `constitution/Constitution.md` §11.4.85; the normative minimums quoted above are what Phase 1 implements.)

What the clause mandates (extracted requirements)

Requirement	§11.4.85 source	Phase 1 coverage
Stress: sustained load N≥100 iters OR ≥30s, with p50/p95/p99	def. 1 (sustained)	YES — `Benchmark/sustained` records every latency, emits percentiles
Stress: concurrent contention N≥10 parallel, all complete, no leak/race	def. 1 (concurrent)	YES — `concurrent` mode, N=64 goroutines, `-race`-clean, FD-count delta recorded
Chaos: fault injection (handler error, dependency unavailable)	def. 2	PARTIAL — error-injection seam wired; Postgres-kill chaos is operator-gated (Phase 1.b)
Chaos: latency injection	def. 2	YES (in-process latency injector middleware)
Chaos: malformed input	def. 2	YES — malformed-body / oversized-path stress variant
Chaos: resource exhaustion (conn-pool, rate limiter trip)	def. 2	PARTIAL — rate-limiter trip recorded; pool-exhaustion is operator-gated (needs real PG)
Recovery assertion on user-visible state after chaos	clause body	YES for in-process chaos; real-dependency recovery is Phase 1.b
Rock-solid proofs / no-bluff (real evidence file, no fabricated numbers)	forensic anchor + §6.J	YES — JSON + MD evidence written from the actual run only
Full automation, re-runnable without manual intervention	§11.4.98 compose	YES — `go test -tags stress` + `scripts/run-chaos-stress.sh`, zero manual steps

Tooling: the clause is outcome-based, not tool-prescriptive. §11.4.85 names no specific load-test tool (no k6, no vegeta, no JMeter). It mandates *outcomes* (recorded latency percentiles, all-complete concurrency, recovery-after-fault on user-visible

state) and a closed-set of *dimensions*. Phase 1 therefore uses the Go stdlib `testing` + `httptest` + goroutines — zero new third-party deps, no sudo, runs on this macOS host. This is the same "reuse the stdlib before adding tooling" posture §11.4.74 prescribes.

---

2. Dimensions

Stress dimensions (load):

- S1 Sustained load — N≥100 sequential requests against the real handler; record p50/p95/p99/max.
- S2 Concurrent contention — N≥10 (Phase 1 uses 64) parallel requests; all must complete; no FD leak; `-race`-clean.

Chaos dimensions (fault):

- C1 Fault injection — handler/dependency returns error mid-load; assert error-rate bounded + recovery after fault clears.
- C2 Latency injection — inject artificial per-request latency; assert the server stays responsive and percentiles degrade gracefully (no hang).
- C3 Dependency kill — kill the Postgres container mid-request stream; assert the API returns a clean 5xx (not a panic/hang) during the outage and recovers (200) after restart. **Operator-gated** (needs podman + real PG; Phase 1.b).
- C4 Resource exhaustion — (a) trip the rate limiter and assert 429 is returned deterministically; (b) exhaust the pgx connection pool and assert graceful backpressure. (a) is in-process now; (b) is operator-gated (Phase 1.b).
- C5 Malformed input — oversized paths, malformed bodies, garbage headers under load; assert no panic, bounded error codes.

---

3. Chosen first target — `lava-api-go`

Decision: the Go API service is the best first stress+chaos target. Justification:

1. **It is the only Lava surface that is a real, long-running server with real dependencies** (Gin HTTP server + Postgres via pgx + cache + rate limiter + rutracker scrape bridge). Stress (sustained/concurrent load) and chaos (dependency kill, pool exhaustion, rate-limiter trip, latency injection) are *native* concepts here. The Android client, by contrast, has no server-side load surface and its chaos analogue (process death, low-memory) needs an emulator — which is §6.X-debt-blocked on this darwin/arm64 host (no `/dev/kvm`/HVF in the podman VM).
2. **It runs on THIS macOS host with no emulator and no sudo.** `httptest.Server` binds a real loopback port in-process; `go test` drives it. The full S1/S2/C1/C2/C4a/C5 set needs nothing but the Go toolchain.
3. **It already has a `tests/load/` directory and a test-type taxonomy** (`tests/{contract,e2e,parity,load,fixtures}`) — the stress+chaos suite slots in as `tests/stress/` alongside, matching the existing convention.
4. **Real evidence is cheap and high-signal here:** latency percentiles, error rates under injected fault, and recovery-time-after-chaos are exactly the "rock-solid proofs" the forensic anchor demands, and they are byte-for-byte reproducible.

The Postgres-kill (C3) and pool-exhaustion (C4b) chaos actions need a real Postgres container under podman. They are **operator-gated** and run via `scripts/run-chaos-stress.sh --with-podman` (Phase 1.b) — NOT faked when the container is absent.

---

4. Evidence each test produces

Every run writes a single JSON evidence file + a human-readable MD companion under `lava-api-go/tests/stress/evidence/<UTC-timestamp>/`. Fields:

- **per-dimension block** with: dimension id (S1/S2/C1…), iterations/concurrency, pass/fail.
- **latency**: p50, p95, p99, max, min, mean (nanoseconds + human ms), histogram bucket counts.
- **error metrics**: total requests, 2xx count, 4xx count, 5xx count, error rate.
- **chaos block**: fault type, fault window (start/end request index), error rate *during* fault, error rate *after* fault clears, recovery-time (requests-until-first-2xx after fault clears).
- **leak guards**: open-FD count before/after (delta must be ~0), goroutine count before/after.
- **provenance**: git SHA (`git rev-parse HEAD`), Go version, GOOS/GOARCH, host, wall-clock.
- **gating verdict**: `PASS`/`FAIL` per the §11.4.85 minimums (N≥100 or ≥30s; N≥10 concurrent; recovery asserted).

No number in the evidence file is ever written unless it came from the actual run. If a dimension is operator-gated and the dependency is absent, the JSON records `"status": "OPERATOR_GATED", "ran": false` — never a fabricated metric (§6.J / §11.4.6).

---

5. §11.4.98 alignment (full automation, re-runnable without manual intervention)

The Phase 1 suite is a plain `go test -tags stress ./tests/stress/...`. It:

- starts its own `httptest.Server` in-process (no external server to boot by hand),
- needs no human action mid-run (no typing, no clicking, no manual webhook),
- runs identically in CI and on the operator's host,
- writes its own evidence file and exits 0/1 on the gating verdict.

The operator-gated Postgres-kill path is *also* full-automation when the flag is supplied — the script brings up the container, runs the chaos, tears it down, with no manual step. It is gated only on the *presence* of podman + the operator's opt-in, not on a manual action during the run.

---

6. Phased plan

Phase 1 (this commit — host-runnable now): `lava-api-go/tests/stress/`

- `stress_harness.go` (build tag `stress`) — the load driver + latency recorder + evidence writer.
- `api_stress_test.go` (build tag `stress`) — S1 sustained, S2 concurrent, C1 in-process fault injection, C2 latency injection, C4a rate-limiter trip, C5 malformed input, against a real Gin handler via `httptest`.
- `scripts/run-chaos-stress.sh` — Lava-side glue: runs the in-process suite by default; `--with-podman` adds C3 (Postgres-kill) + C4b (pool-exhaustion) when the operator opts in.

Phase 1.b (operator-gated, same code, needs podman + real PG): C3 + C4b via the script's `--with-podman` path. Reuses the existing `lava-api-go` podman compose for the DB.

Phase 2 (deferred — Android/emulator side): process-death + low-memory + airplane-mode + slow-network chaos for the Compose client, driven through the §6.X containerized emulator matrix. Blocked on the standing §6.X-debt (darwin/arm64 host has no KVM/HVF in the podman VM). Will land on a Linux x86_64 gate-host. Documented honestly as deferred, not faked.

Phase 3 (deferred): extend the stress+chaos pattern to the Ktor `:proxy` and to the rutracker scrape bridge (real-tracker chaos: upstream-slow, upstream-503, Cloudflare-challenge), gated by `-PrealTrackers=true` so default runs make no outbound calls.

---

7. §11.4.74 catalogue-check (reuse-before-build)

Lava already vendors resilience-relevant submodules. Confirmed from each submodule's own `CLAUDE.md` / `README.md`:

- **`submodules/recovery/`** (`digital.vasic.recovery`) — `pkg/breaker` (named circuit breakers + registry), `pkg/health` (periodic health checker with `Status` tracking), `pkg/facade` (`Resilience` API: `Execute`, `GetOrCreateBreaker`, `AddHealthCheck`, `Stats`, `Stop`). This is the **production** resilience surface the C3 dependency-kill chaos test will drive in Phase 1.b — the circuit-breaker + health-checker are exactly what make a Postgres-outage recover cleanly.
- **`submodules/ratelimiter/`** — production rate-limiting (the C4a dimension asserts its 429-under-load *semantics*; the in-process Phase-1 version proves the harness can detect a deterministic 429, while the real limiter is exercised by lava-api-go's existing e2e suite via `internal/ratelimit/`).
- **`submodules/concurrency/`** — `pkg/breaker` engine (the lower-level breaker `recovery` composes) + `pkg/safe` concurrent-safe containers.
- **`submodules/observability/`** — `RecordNonFatal` telemetry surface (§6.AC).

Reuse-vs-build call: the Phase 1 **stress driver itself** (load generation + latency percentile recording + evidence writing) is *test infrastructure*, not a production capability — no submodule provides a Go load-generator / percentile-recorder, so it

is built here in `tests/stress/` (the correct home for test glue, per §11.4.74 "no-match"). The **production** capabilities the chaos tests *exercise* (circuit-breaking + health-checking via `recovery`, rate limiting via `ratelimiter`, telemetry via `observability`) are already provided by the submodules and are NOT reimplemented — the stress suite *drives* them, it does not duplicate them. Phase 1.b C3/C4b will import `digital.vasic.recovery/pkg/facade` (via lava-api-go's existing `internal/` glue) rather than re-roll a breaker.

`Catalogue-Check:` reuse `vasic-digital/recovery` + `vasic-digital/ratelimiter` + `vasic-digital/observability` (chaos *targets*); no-match for the load-driver test glue (built in `tests/stress/`).

---

8. Honest status of this investigation

- §11.4.85 text: **CONFIRMED verbatim** (quoted in §1) from both the pinned tree and `origin/main`.
- lava-api-go layout + test taxonomy: **CONFIRMED** — `tests/{contract,e2e,parity,load,qa,compose,integration,scripts,fixtures}` + a new `tests/stress/` added by this work; `internal/{archiveorg,auth,cache,config,discovery,firebase,gen,gutenberg,handlers,kinozal,middleware,nnmclub,observability,provider,qa,ratelimit,rutracker,server,version}`.
- Resilience submodule APIs: **CONFIRMED** from each submodule's own `CLAUDE.md`/`README.md` (see §7) — `recovery` exposes `pkg/{breaker,health,facade}`; reuse-vs-build call recorded.
- Phase 1 suite: **RAN, verdict PASS, EXIT=0** — see `EVIDENCE-phase1.md` for verbatim numbers.
- Constitution pin: **NOT bumped** (CONST-049 / operator-gated). The pin is `208e2c8`; §11.4.85 is present in both the pinned tree and `origin/main` (`883ccc1`) at the same line offsets.